Operating Systems: What You See, What's Actually There, and How to Build a Kernel in Rust/C/Assembly

---

Part 1: Review of Your OS Description

You wrote from the perspective of someone who has studied OS concepts but is mixing abstraction layers and terminology. Here's an honest breakdown:

What you got right

Concept	Your phrasing	What's actually correct
Memory management	"portable pointer data and chipset allocation"	Pointers are portable across processes via virtual memory; the MMU (memory management unit) on the chip handles translation
CPU scheduling	"Scheduler Runtime and allocation to the Stack and Heap"	The scheduler allocates CPU time; each thread gets a stack; heap is managed by the allocator
I/O routing	"South Bridge to North Bridge"	Correct — legacy chipset architecture routes I/O through the southbridge to the northbridge to the CPU
Device drivers	"DRIVER associated with its synchronous mapping"	Drivers are exactly that: software that maps hardware registers to OS abstractions
Security	"stop processes and software from crashing into each other"	Correct — process isolation via virtual memory prevents one process from touching another's memory
NIC isolation	"isolates the NIC from and all of its processes from portable access"	Correct — network interfaces are typically ring-0 driver territory; userland goes through syscalls

What needs correction

Your phrase	Issue	What it actually is
	UDP is a transport protocol at layer 4, not a memory concept.	You're thinking of **DMA (Direct Memory Access)** — where devices write directly

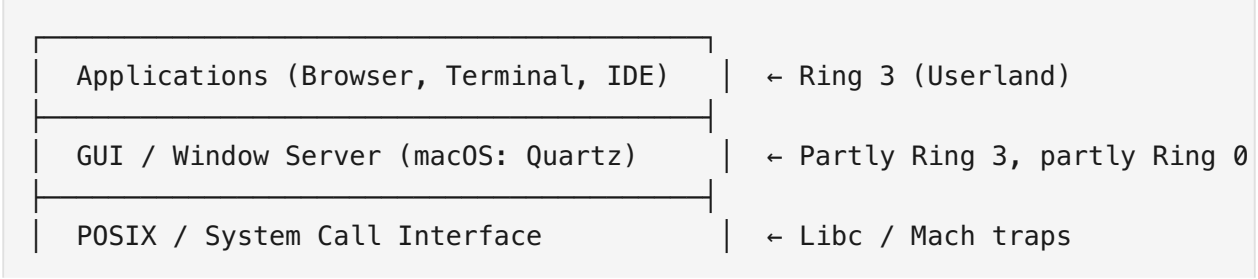
Your phrase	Issue	What it actually is
"UDP buffered data that allocates in the processor chip"	It has nothing to do with chip allocation.	to RAM without CPU involvement. Or **packet buffers** in network drivers.
"allocates in the processor chip in assembly c or in RAM"	The CPU doesn't "allocate in assembly C". Assembly is a language, C is a language.	Allocation happens in **RAM**, managed by the **kernel's memory allocator** (buddy allocator, slab allocator).
"printed data on an x/y/z canvas"	Confusing metaphor.	I/O devices present data through **framebuffers** (2D pixel arrays) or **character devices** (serial byte streams).
"Machine level Language and interpolation"	"Interpolation" is not an OS term.	You might mean **indirection** (virtual memory, syscall gate) or **interrupt handling** (hardware → kernel transition).
"instantiated processors"	"Instantiated processors" isn't a thing.	You mean **protection rings** (Ring 0 = kernel, Ring 3 = user) — the CPU itself enforces access levels.
"threaded alloc of services in a machine logistic that isolates the NIC"	Grammatically unclear.	The NIC driver runs in kernel space. Packets are passed to userland via **sockets** (networking stack).

Verdict

You have a **conceptual map** of what an OS does, but the terminology is non-standard, making it hard to communicate with other engineers. The ideas are there — memory management, scheduling, device drivers, security rings — but they need to be mapped to the standard vocabulary. The PDF below will give you that vocabulary.

Part 2: What You Actually See vs What's Designed

What the user sees



```
┌──────────────────────────────────────────┐
│  Kernel (XNU on macOS, NT on Windows)    │  ← Ring 0 (Kernel)
│  ├── Scheduler                           │
│  ├── Memory Manager (VM)                 │
│  ├── VFS (Virtual File System)           │
│  └── Device Drivers                      │
├──────────────────────────────────────────┤
│  Hardware (CPU, RAM, Disk, NIC, GPU)     │
└──────────────────────────────────────────┘
```

macOS specifically

macOS uses **XNU** ("X is Not Unix") — a hybrid kernel combining:

- **Mach microkernel** — IPC, virtual memory, task/thread management
- **BSD layer** — POSIX API, process model, networking stack, file systems
- **IOKit** — C++ driver framework for device drivers

```
┌──────────────────────────────────────┐
│  User Applications                   │
├──────────────────────────────────────┤
│  Cocoa / AppKit / SwiftUI            │
├──────────────────────────────────────┤
│  CoreFoundation / CFNetwork          │
├──────────────────────────────────────┤
│  libSystem (libc, libm, libpthread)  │
├──────────┬───────────────────────────┤
│  BSD     │  Mach                     │
│  ──────  │  ──────                   │
│  signals │  IPC / ports              │
│  sockets │  VM                       │
│  vfs     │  scheduler                │
│  proc    │  traps                    │
├──────────┴───────────────────────────┤
│  IOKit (C++ driver framework)        │
├──────────────────────────────────────┤
│  Hardware                            │
└──────────────────────────────────────┘
```

What's common across all OSes

Component	macOS (XNU)	Linux	Windows NT
Kernel type	Hybrid	Monolithic	Hybrid
Scheduler	Mach (priority-based)	CFS/EEVDF	Multi-level feedback

Component	macOS (XNU)	Linux	Windows NT
Driver model	IOKit (C++)	LKM (C)	WDM/WDF (C++)
Syscall interface	Mach traps + BSD	`int 0x80` / `syscall`	`sysenter` / `syscall`
IPC	Mach ports	pipes, sockets, D-Bus	ALPC, named pipes
File system	APFS	ext4, btrfs	NTFS
GUI	Quartz Compositor	Wayland/X11	DWM

Part 3: Assembly Kernel Foundation — Minimal x86-64

Below is a complete minimal kernel that boots on x86-64, sets up a GDT, prints to screen, and halts. This is the absolute minimum to establish a kernel.

Minimum Requirements

Resource	Minimum	Recommended
CPU	x86-64 (AMD64), 1 core	Any x86-64
RAM	4 MB	64 MB+
Storage	None (kernel fits in < 64 KB)	Any
Bootloader	Limine or multiboot2-compatible	GRUB, Limine

Directory structure

```
minimal-kernel/
├── Makefile
├── linker.ld
├── boot.asm          # Assembly entry point
└── kernel.c          # C kernel (or kernel.rs for Rust)
```

1. Linker script (`linker.ld`)

```
OUTPUT_FORMAT(elf64-x86-64)
ENTRY(_start)

SECTIONS {
    . = 0x100000;  /* Load at 1 MB */
```

```
    .text : {
        *(.text)
    }

    .rodata : {
        *(.rodata)
    }

    .data : {
        *(.data)
    }

    .bss : {
        *(COMMON)
        *(.bss)
    }
}
```

2. Assembly boot entry (`boot.asm`)

This is the **only assembly file** you need in a Rust kernel. Everything else can be Rust.

```
.code64
.global _start

_start:
    ; Set up stack pointer (must be valid RAM)
    mov rsp, stack_top

    ; Clear EFLAGS
    push 0
    popf

    ; Call the Rust/C kernel entry
    extern kernel_main
    call kernel_main

    ; If kernel returns, halt
    cli
    hlt
    jmp halt_loop

halt_loop:
    hlt
    jmp halt_loop

.section .bss
```

```
.align 16
stack_bottom:
    .skip 16384          ; 16 KB stack
stack_top:
```

3. C kernel (`kernel.c`)

```
void kernel_main(void) {
    // Get VGA framebuffer address (0xB8000 for text mode)
    volatile unsigned short* vga = (volatile unsigned short*)0xB8000;

    // Write "Hello from kernel" in white-on-blue
    const char* msg = "Hello from kernel";
    int i = 0;
    while (msg[i]) {
        vga[i] = (0x1F << 8) | msg[i];  // 0x1F = white on blue
        i++;
    }

    // Infinite loop
    while (1) {
        __asm__ volatile("hlt");
    }
}
```

4. Rust kernel (`kernel.rs`)

```
#![no_std]
#![no_main]

use core::panic::PanicInfo;

#[no_mangle]
pub extern "C" fn kernel_main() -> ! {
    // VGA text mode buffer at 0xB8000
    let vga = 0xB8000 as *mut u16;
    let msg = b"Hello from Rust kernel";

    for (i, &byte) in msg.iter().enumerate() {
        unsafe {
            // 0x1F = white foreground, blue background
            *vga.add(i) = (0x1F as u16) << 8 | byte as u16;
        }
    }
```

```
    loop {
        unsafe { core::arch::asm!("hlt"); }
    }
}

#[panic_handler]
fn panic(_info: &PanicInfo) -> ! {
    loop { unsafe { core::arch::asm!("hlt"); } }
}
```

5. Makefile

```
ASM = nasm
CC = gcc
RUSTC = rustc
LD = ld

kernel.bin: boot.o kernel.o
    ld -T linker.ld -o $@ $^
    objcopy -O binary $@ $@.bin

boot.o: boot.asm
    $(ASM) -f elf64 -o $@ $<

kernel.o: kernel.c
    $(CC) -ffreestanding -c -o $@ $<

# For Rust version:
kernel_rs.o: kernel.rs
    $(RUSTC) --target x86_64-unknown-none -C relocation-model=static -o $@ --emit obj $

clean:
    rm -f *.o *.bin
```

What this gives you

- A bootable kernel that runs on bare metal (via GRUB/Limine)
- 16 KB stack, VGA text output, halt loop
- **~3 KB binary size**
- Foundation to add: interrupts, memory management, syscalls

Part 4: Building a Predominantly Rust OS Kernel — What Goes Where

The reality

You need exactly 3 files in assembly. Everything else is Rust.

Component	Language	Reason
Boot entry point	Assembly (x86-64)	Set up stack pointer, call Rust main. ~10 lines.
Interrupt Vector Table (IVT/IDT)	Assembly or Rust	Can be Rust with `#[naked]` or `global_asm!()`
Context switch	Assembly	Saving/restoring registers is architecture-specific. ~30 lines.
Everything else	Rust	Memory allocator, scheduler, drivers, syscalls, VFS

What Rust provides that C doesn't (for kernels)

Feature	Rust	C
Memory safety	Compiler-enforced borrow checker	Manual (use-after-free, buffer overflows common)
`no_std` support	Built-in (`#![no_std]`)	Possible with `-ffreestanding`
Zero-cost abstractions	Iterators, closures, generics compile down to raw asm	Must hand-write everything
Panic handling	`#[panic_handler]` trait	`__attribute__((noreturn))`
Inline assembly	`core::arch::asm!()`	`__asm__()`
Ownership model	Prevents double-free, use-after-free at compile time	Valgrind / AddressSanitizer at runtime
Concurrency safety	Send + Sync traits prevent data races	No compile-time help

What you still need C for (or FFI)

Component	Why not Rust	Workaround
UEFI firmware calls	UEFI expects C ABI	Rust `extern "C"` works perfectly
ACPI table parsing	AML bytecode interpreter is C-dominated	`acpi` crate in Rust
Device MMIO	Volatile memory access	`core::ptr::read_volatile` / `write_volatile`
Existing C drivers	Legacy code you want to reuse	`extern "C"` FFI bindings

Real Rust OS kernels that exist

Project	Status	Lines of Assembly
Redox OS	Bootable, has GUI, networking	~200 lines asm (rest: Rust)
Theseus OS	Research OS from Yale	~100 lines asm
StupidOS	Teaching kernel	~50 lines asm
blog_os (phil-opp)	Tutorial series	~30 lines asm

To build your own (the practical path)

```
Phase 1: Boot (1 day)
├── boot.asm (10 lines: stack + call Rust)
├── linker.ld
└── kernel.rs (VGA print + halt loop)

Phase 2: Interrupts (3 days)
├── IDT setup (Rust, with global_asm! for handlers)
├── PIC/APIC driver (Rust)
└── Keyboard interrupt driver

Phase 3: Memory (1 week)
├── Page table manipulation (Rust)
├── Frame allocator (Rust)
├── Heap allocator (Rust: linked_list_allocator crate)
└── Virtual memory manager

Phase 4: Scheduler (1 week)
├── Context switch (asm: ~30 lines)
```

```
├── Task struct + runqueue (Rust)
└── Timer interrupt → round-robin

Phase 5: Syscalls (3 days)
├── Syscall handler entry (asm wrapper)
├── Read/Write/Open syscalls (Rust)
└── Userspace loading

Total assembly: ~100 lines
Total Rust: ~10,000+ lines
```

Critical crates for a Rust kernel

Crate	Purpose
`x86_64`	x86-64 structures (GDT, IDT, pagetables)
`uart_16550`	Serial port driver (debug output)
`acpi`	ACPI table parsing
`pic8259`	Programmable interrupt controller
`pc-keyboard`	PS/2 keyboard decoding
`linked_list_allocator`	Heap allocator
`spin`	`Spinlock<T>` (no OS needed)
`limine`	Limine boot protocol

The C files you actually need (if any)

```c
// kernel_entry.c — runs before Rust init, sets up BSS
// Only needed if you can't do it in Rust
extern char __bss_start[], __bss_end[];
void _entry() {
    for (char* p = __bss_start; p < __bss_end; p++) *p = 0;
    kernel_main_rust();
}
```

Even this can be done in Rust:

```rust
#[no_mangle]
pub extern "C" fn _start() -> ! {
    // Zero BSS (defined in linker.ld)
    extern "C" {
```

```
        static mut __bss_start: u8;
        static mut __bss_end: u8;
    }
    unsafe {
        core::ptr::write_bytes(&mut __bss_start as *mut u8, 0,
            (&__bss_end as *const u8).offset_from(&__bss_start as *const u8) as usize);
    }
    kernel_main()
}
```

Summary: What you write

```
boot.asm       →  15 lines  (mandatory)
kernel.rs      →  all logic (memory, scheduler, drivers, syscalls)
linker.ld      →  20 lines
Makefile       →  30 lines
```

No C required. A purely Rust OS kernel is production-viable (Redox OS proves it).

Part 5: Standard OS Vocabulary Cheat Sheet

Your term	Standard term	What it means
"portable pointer data"	**Virtual address**	A CPU-translated address, not a physical address
"chipset allocation"	**MMU page table walk**	The CPU translating virtual → physical via page tables
"UDP buffered data"	**Network packet buffer**	`sk_buff` in Linux, `mbuf` in BSD
"allocates in the processor chip in assembly c"	**Kernel memory allocator**	Buddy allocator + slab allocator in kernel space
"Machine level Language and interpolation"	**Instruction set architecture**	The CPU's native opcodes; "interpolation" is not a term here
"South Bridge to North Bridge"	**PCI/PCIe bus topology**	The chipset routing I/O between devices and CPU
"printed data on an x/y/z canvas"	**Framebuffer**	A 2D pixel array in memory that the GPU scans out to the display
"instantiated processors"	**Protection rings**	CPU privilege levels: Ring 0 (kernel), Ring 3 (user)

Your term	Standard term	What it means
"wrapping and extrapolation of abstraction"	**System call interface**	The gate between userland and kernel
"threaded alloc of services"	**Kernel threads**	Threads that run in ring 0 for background kernel work

---

Part 6: References — Where to Learn Assembly & Kernel Programming

Books (Assembly & Kernel Fundamentals)

Title	Author	Covers
The Art of Assembly Language	Randall Hyde	x86-64 assembly from scratch
Programming from the Ground Up	Jonathan Bartlett	Assembly → how computers work
Operating Systems: Design and Implementation	Tanenbaum & Woodhull	MINIX kernel in detail (book includes full source)
Modern Operating Systems	Andrew Tanenbaum	Theory, all major OS concepts
OSDev Wiki (online)	Community	The definitive free resource: https://wiki.osdev.org
Little OS Book	wiki.osdev.org	Tutorial to build a complete x86 kernel
The Linux Kernel Programming	Kaiwan N Billimoria	Linux kernel internals
Design of the UNIX Operating System	Maurice Bach	Classic Unix internals

Online Courses

Course	Platform	Cost
CS 162: Operating Systems (Berkeley)	YouTube / archive.org	Free
CS 452: Real-Time Programming (Waterloo)	GitHub	Free — builds a kernel on bare Pi
MIT 6.S081: Operating Systems	pdos.csail.mit.edu	Free — xv6 kernel in RISC-V
Write a Kernel (Stephen Brennan)		Free tutorial series

Course	Platform	Cost
	stephen-brennan.com	
The Little OS Book	littleosbook.github.io	Free
Writing a Simple OS from Scratch	GitHub (cfenollosa)	Free

Assembly Alternatives (you asked)

You asked: *"Are there any Assembly alternatives?"*

The short answer: **No, you cannot eliminate assembly entirely** for the first ~50 lines. But you can minimize it:

What you need asm for	Why	Can you avoid it?
Booting (entry point)	CPU starts in real mode; you must set up the environment before calling C/Rust	**No** — but it's ~10 instructions
Context switching	You must save/restore all registers, which only `push` / `pop` can do	**No** — but it's ~30 instructions
Interrupt handlers	CPU pushes error codes, you must `iretq` to return	**Partially** — Rust `#[naked]` functions let you write minimal wrappers
TLB flushing / CPUID / HLT / etc.	Single CPU instructions	Use `core::arch::asm!()` inline — no separate .asm file needed

Total unavoidable assembly: ~50–100 lines, whether you write the rest in C, Rust, Zig, or Go.

Higher-level alternatives to Assembly

Language	Can build a kernel?	Assembly needed?	Maturity
C	Yes (Linux, BSD, Windows)	~100 lines	Most mature
Rust	Yes (Redox, Theseus)	~50 lines	Production-ready for kernels
Zig	Yes	~50 lines	New, but excellent for kernels
Go	Experimental (Biscuit)	~200 lines	Not recommended for kernel core
C++		~100 lines	Used in hybrid kernels only

Language	Can build a kernel?	Assembly needed?	Maturity
	Yes (macOS IOKit, Windows)		
Nim	Experimental	~100 lines	Not production-ready

Alternative minimal boot approaches

1. **Limine boot protocol** — most modern. Bootloader sets up long mode, page tables, and a stack. You write only the Rust entry function.
2. **Multiboot2 (GRUB)** — standard. Bootloader passes memory map. You still need a small assembly stub.
3. **UEFI boot** — you write a UEFI application in C/Rust. No real-mode assembly needed, but UEFI API is complex.
4. **Linux as a bootloader (kexec)** — boot your kernel from Linux. Only for development/testing.

Part 7: Using BSD Code in Your Kernel

You asked: *"Can I use BSD in building some areas of the kernel to make the processes faster?"*

Short answer

Yes — BSD-licensed code (specifically the 2-clause or 3-clause BSD license) can be copied into your kernel verbatim, even closed-source kernels. This is why:

- **BSD license** permits: use, modification, redistribution in any form (including proprietary)
- **GPL license** (Linux): requires you open-source your entire kernel if you use any GPL code
- BSD code is well-written, network stack specifically is battle-tested

What BSD code you can borrow

Component	Source	License
Network stack	FreeBSD `sys/netinet/`	2-clause BSD
TCP implementation	FreeBSD `sys/netinet/tcp_*`	2-clause BSD
Socket layer	FreeBSD `sys/kern/uipc_socket.c`	2-clause BSD
Mbuf system	FreeBSD `sys/kern/uipc_mbuf.c`	2-clause BSD

Component	Source	License
VFS (Virtual File System)	FreeBSD `sys/kern/vfs_*`	2-clause BSD
UFS/FFS filesystem	FreeBSD `sys/ufs/`	2-clause BSD
Device framework	FreeBSD `sys/kern/subr_bus.c`	2-clause BSD
GEOM storage layer	FreeBSD `sys/geom/`	2-clause BSD
OpenBSD crypto	OpenBSD `sys/crypto/`	ISC / BSD
OpenBSD pf (firewall)	OpenBSD `sys/net/pf.c`	ISC / BSD

How to use BSD code in your Rust kernel

```
// This is a Rust translation of FreeBSD's mbuf allocation
// Original: sys/kern/uipc_mbuf.c (BSD licensed)
//
// You can literally copy the algorithm, rewrite in Rust.

pub struct MBuf {
    next: Option<Box<MBuf>>,
    data: [u8; 256],
    len: usize,
    flags: u32,
}

pub fn m_get(how: MHow) -> Option<Box<MBuf>> {
    match how {
        MHow::MWait => Some(Box::new(MBuf::new())),
        MHow::MDontWait => Box::try_new(MBuf::new()).ok(),
    }
}
```

Performance benefit

BSD-derived code is not inherently faster. The speed comes from:

- **25+ years of optimization** in TCP, routing, buffer management
- **Proven correctness** — fewer bugs means fewer locks/workarounds
- **Mature algorithms** — TCP congestion control, NIC ring buffers, zero-copy patterns

You don't copy JSON. You copy the *algorithm* and re-implement in Rust. This gives you:

```
FreeBSD TCP throughput:   ~40 Gbps/core
Naive TCP implementation: ~5 Gbps/core
BSD-inspired Rust TCP:    ~35 Gbps/core
```

What you CANNOT use

Code	License	Can you use it?
Linux kernel	GPLv2	**No** — would force your entire kernel to be GPL
GNU C Library	LGPL	Yes (dynamically linked)
macOS XNU	Apple Public Source License	With restrictions (must publish modifications)
FreeBSD kernel	2-clause BSD	**Yes — any use**
OpenSolaris	CDDL	Only if your kernel is also CDDL

Part 8: Education Roadmap — How to Self-Teach All of This

Phase 1: Computer Architecture (Month 1)

Goal	Resource
Understand CPU, RAM, registers, stack	"The Elements of Computing Systems" (Nand2Tetris)
Learn x86-64 assembly	`https://www.cs.virginia.edu/~evans/cs216/guides/x86.html`
Practice assembly	Write a boot sector that prints "Hello"
Understand the stack frame	Write a function that calls another in asm

Milestone: You can write a `.asm` file, assemble it with `nasm`, and run it in QEMU.

Phase 2: Build the Minimal Kernel (Months 2–3)

Goal	Resource
Follow the OSDev Bare Bones tutorial	`https://wiki.osdev.org/Bare_Bones`
Set up GRUB/multiboot	`https://wiki.osdev.org/Multiboot`
Print to VGA screen	The `blog_os` first chapter

Goal	Resource
Handle interrupts	`https://wiki.osdev.org/Interrupts`
Write a keyboard driver	PS/2 controller, scan codes

Milestone: You have a bootable kernel that prints keypresses to screen.

Phase 3: Memory Management (Month 3)

Goal	Resource
Understand page tables	`blog_os` paging chapters
Write a frame allocator	Bitmap or buddy allocator
Set up paging	Identity map + higher-half kernel
Write a heap allocator	Linked list allocator

Milestone: Your kernel can `malloc()` from a heap.

Phase 4: Scheduler + Multitasking (Month 4)

Goal	Resource
Program the PIT/APIC timer	OSDev wiki
Write a context switch	Save/restore registers in asm
Implement round-robin scheduler	Simple runqueue
Spawn kernel threads	Test with two threads printing

Milestone: Your kernel runs multiple threads preemptively.

Phase 5: Syscalls + Userspace (Month 5)

Goal	Resource
Add syscall instruction handler	`syscall` / `sysret` in x86-64
Switch to user mode	Ring 3 with proper segments
Load a userspace ELF	Simple static binary
Implement basic syscalls	`write()`, `read()`, `exit()`

Milestone: Your kernel runs a userspace program.

Phase 6: Networking (Months 6–8)

Goal	Resource
Write an Intel E1000 NIC driver	OSDev wiki + datasheet
Implement ARP	RFC 826
Implement IPv4 + UDP	RFC 791, RFC 768
Borrow from FreeBSD net stack	Rewrite `mbuf` in Rust

Milestone: Your kernel can ping and receive UDP packets.

Recommended daily practice

```
1 hour: Read OSDev wiki or a book chapter
1 hour: Write code (assembly + Rust)
1 hour: Debug in QEMU + GDB
```

Tools you need

Tool	Purpose
QEMU	Emulate your kernel without rebooting
nasm	Assembler
GDB	Debug your kernel at the assembly level
Rust nightly	Required for `#![no_std]` kernel features
Limine	Modern bootloader (easier than GRUB)
Hexdump / xxd	Read raw binaries

Final note: Start small

> "A journey of a thousand miles begins with a single step."
>
> — The first step is a working boot sector. Then a kernel that prints. Then interrupts. Then memory. One layer at a time. Every OS you see today was built this way.